Two Dimensional Binary Indexed Tree or Fenwick Tree

Last Updated : 30 Nov, 2023

Prerequisite - Fenwick Tree

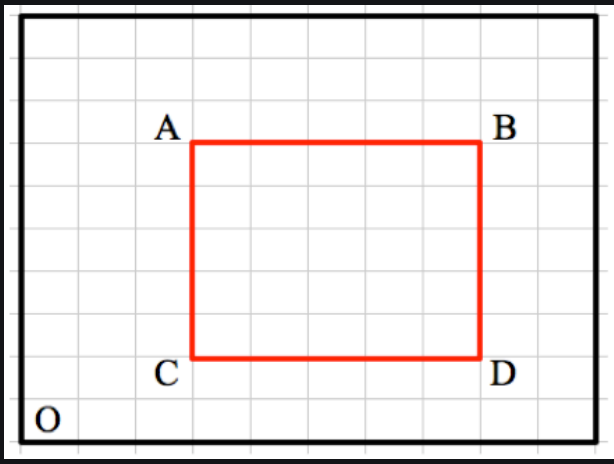
We know that to answer range sum queries on a 1-D array efficiently, binary indexed tree (or Fenwick Tree) is the best choice (even better than segment tree due to less memory requirements and a little faster than segment tree).

Can we answer sub-matrix sum queries efficiently using Binary Indexed Tree ?

The answer is **yes**. This is possible using a **2D BIT** which is nothing but an array of 1D BIT.

Algorithm:

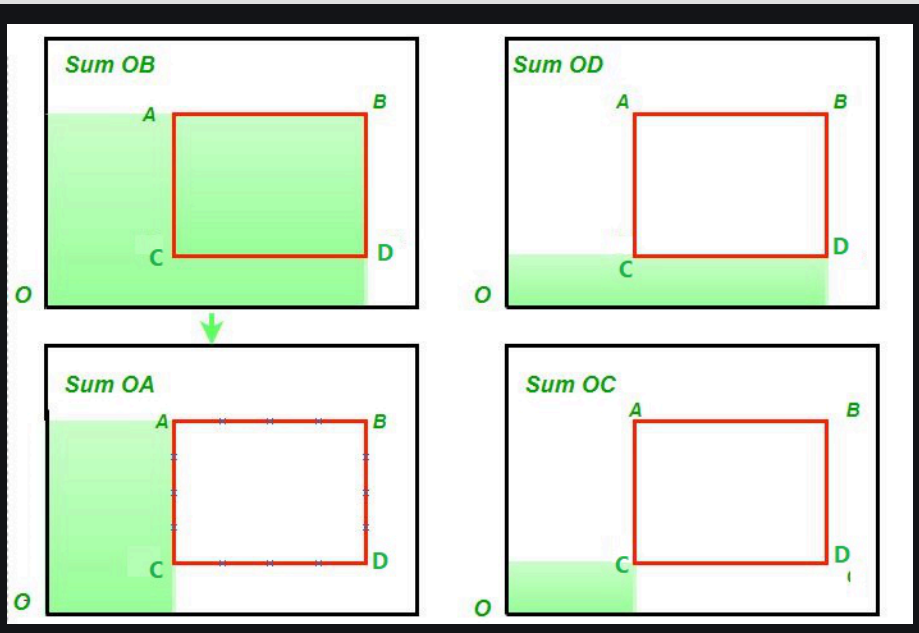
We consider the below example. Suppose we have to find the sum of all numbers inside the highlighted area:



70	37	23	57	27	22	90	99	22	59
47	63	33	1	42	46	6	70	98	93
36	62	50	21	92	27	60	29	15	34
53	3	88	45	57	39	83	81	79	56
28	63	89	20	47	15	84	18	82	33
26	87	11	76	79	5	94	55	73	51
17	82	86	10	96	5	42	43	51	6
44	76	51	4	15	99	52	11	70	89
66	36	92	85	50	21	72	27	52	65
60	0	67	37	59	14	33	13	36	36

We assume the origin of the matrix at the bottom - O.Then a 2D BIT exploits the fact that-

```
Sum under the marked area = Sum(OB) - Sum(OD) -
                            Sum(OA) + Sum(OC)
```



In our program, we use the getSum(x, y) function which finds the sum of the matrix from (0, 0) to (x, y).
Hence the below formula :

```
Sum under the marked area = Sum(OB) - Sum(OD) -
                            Sum(OA) + Sum(OC)

The above formula gets reduced to,

Query(x1,y1,x2,y2) = getSum(x2, y2) -
                     getSum(x2, y1-1) -
                     getSum(x1-1, y2) +
                     getSum(x1-1, y1-1)
```

where,
x1, y1 = x and y coordinates of C
x2, y2 = x and y coordinates of B
The updateBIT(x, y, val) function updates all the elements under the

region – (x, y) to (N, M) where,

N = maximum X co-ordinate of the whole matrix.

M = maximum Y co-ordinate of the whole matrix.

The rest procedure is quite similar to that of 1D Binary Indexed Tree.

Below is the C++ implementation of 2D indexed tree

C++ Java Python3 C# JavaScript

```
/* C++ program to implement 2D Binary Indexed Tree

2D BIT is basically a BIT where each element is another BIT.
Updating by adding v on (x, y) means it's effect will be found
throughout the rectangle [(x, y), (max_x, max_y)],
and query for (x, y) gives you the result of the rectangle
[(0, 0), (x, y)], assuming the total rectangle is
[(0, 0), (max_x, max_y)]. So when you query and update on
this BIT,you have to be careful about how many times you are
subtracting a rectangle and adding it. Simple set union formula
works here.

So if you want to get the result of a specific rectangle
[(x1, y1), (x2, y2)], the following steps are necessary:

Query(x1,y1,x2,y2) = getSum(x2, y2)-getSum(x2, y1-1) -
                     getSum(x1-1, y2)+getSum(x1-1, y1-1)

Here 'Query(x1,y1,x2,y2)' means the sum of elements enclosed
in the rectangle with bottom-left corner's co-ordinates
(x1, y1) and top-right corner's co-ordinates - (x2, y2)

Constraints -> x1<=x2 and y1<=y2

    /\
 y  |
    |          --------(x2,y2)
    |         |       |
    |         |       |
    |         |       |
    |         ---------
```

```
    |       (x1,y1)
    |
    |_____________________________
   (0, 0)                    x-->

In this program we have assumed a square matrix. The
program can be easily extended to a rectangular one. */

#include<bits/stdc++.h>
using namespace std;

#define N 4 // N-->max_x and max_y

// A structure to hold the queries
struct Query
{
    int x1, y1; // x and y co-ordinates of bottom left
    int x2, y2; // x and y co-ordinates of top right
};

// A function to update the 2D BIT
void updateBIT(int BIT[][N+1], int x, int y, int val)
{
    for (; x <= N; x += (x & -x))
    {
        // This loop update all the 1D BIT inside the
        // array of 1D BIT = BIT[x]
        for (int yy=y; yy <= N; yy += (yy & -yy))
            BIT[x][yy] += val;
    }
    return;
}

// A function to get sum from (0, 0) to (x, y)
int getSum(int BIT[][N+1], int x, int y)
{
    int sum = 0;

    for(; x > 0; x -= x&-x)
    {
```

```
        // This loop sum through all the 1D BIT
        // inside the array of 1D BIT = BIT[x]
        for(int yy=y; yy > 0; yy -= yy&-yy)
        {
            sum += BIT[x][yy];
        }
    }
    return sum;
}

// A function to create an auxiliary matrix
// from the given input matrix
void constructAux(int mat[][N], int aux[][N+1])
{
    // Initialise Auxiliary array to 0
    for (int i=0; i<=N; i++)
        for (int j=0; j<=N; j++)
            aux[i][j] = 0;

    // Construct the Auxiliary Matrix
    for (int j=1; j<=N; j++)
        for (int i=1; i<=N; i++)
            aux[i][j] = mat[N-j][i-1];

    return;
}

// A function to construct a 2D BIT
void construct2DBIT(int mat[][N], int BIT[][N+1])
{
    // Create an auxiliary matrix
    int aux[N+1][N+1];
    constructAux(mat, aux);

    // Initialise the BIT to 0
    for (int i=1; i<=N; i++)
        for (int j=1; j<=N; j++)
            BIT[i][j] = 0;

    for (int j=1; j<=N; j++)
```

```
    {
        for (int i=1; i<=N; i++)
        {
            // Creating a 2D-BIT using update function
            // everytime we/ encounter a value in the
            // input 2D-array
            int v1 = getSum(BIT, i, j);
            int v2 = getSum(BIT, i, j-1);
            int v3 = getSum(BIT, i-1, j-1);
            int v4 = getSum(BIT, i-1, j);

            // Assigning a value to a particular element
            // of 2D BIT
            updateBIT(BIT, i, j, aux[i][j]-(v1-v2-v4+v3));
        }
    }

    return;
}

// A function to answer the queries
void answerQueries(Query q[], int m, int BIT[][N+1])
```

tions Quizzes Must Do Advanced DSA System Design Aptitude Puzzles I Sign In

```
    for (int i=0; i<m; i++)
    {
        int x1 = q[i].x1 + 1;
        int y1 = q[i].y1 + 1;
        int x2 = q[i].x2 + 1;
        int y2 = q[i].y2 + 1;

        int ans = getSum(BIT, x2, y2)-getSum(BIT, x2, y1-1)-
                  getSum(BIT, x1-1, y2)+getSum(BIT, x1-1, y1-
1);

        printf ("Query(%d, %d, %d, %d) = %d\n",
                q[i].x1, q[i].y1, q[i].x2, q[i].y2, ans);
    }
    return;
}

// Driver program
```

```
int main()
{
    int mat[N][N] = {{1, 2, 3, 4},
                     {5, 3, 8, 1},
                     {4, 6, 7, 5},
                     {2, 4, 8, 9}};

    // Create a 2D Binary Indexed Tree
    int BIT[N+1][N+1];
    construct2DBIT(mat, BIT);

    /* Queries of the form - x1, y1, x2, y2
       For example the query- {1, 1, 3, 2} means the sub-
matrix-
    y
    /\
 3  |       1 2 3 4      Sub-matrix
 2  |       5 3 8 1      {1,1,3,2}      --->     3 8 1
 1  |       4 6 7 5                                6 7 5
 0  |       2 4 8 9
    |
  --|------ 0 1 2 3 ----> x
    |

    Hence sum of the sub-matrix = 3+8+1+6+7+5 = 30

    */

    Query q[] = {{1, 1, 3, 2}, {2, 3, 3, 3}, {1, 1, 1, 1}};
    int m = sizeof(q)/sizeof(q[0]);

    answerQueries(q, m, BIT);

    return(0);
}
```

Output

```
Query(1, 1, 3, 2) = 30
Query(2, 3, 3, 3) = 7
```

```
Query(1, 1, 1, 1) = 6
```

Time Complexity:

- Both updateBIT(x, y, val) function and getSum(x, y) function takes O(log(N)*log(M)) time.
- Building the 2D BIT takes O(NM log(N)*log(M)).
- Since in each of the queries we are calling getSum(x, y) function so answering all the Q queries takes **O(Q*log(N)*log(M))** time.
- Hence the overall time complexity of the program is **O((NM+Q)*log(N)*log(M))** where,
  N = maximum X co-ordinate of the whole matrix.
  M = maximum Y co-ordinate of the whole matrix.
  Q = Number of queries.

Auxiliary Space: O(NM) to store the BIT and the auxiliary array

Comment

T tech manager 14

Article Tags : Misc Advanced Data Structure DSA array-range-queries

+1 More

Explore

DSA Fundamentals

Data Structures

Algorithms

Advanced

Interview Preparation

Practice Problem

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play
Download on the App Store

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Campus Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
IBM Certification
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA

Preparation Corner
Interview Corner

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved